

# Fase V1: Del Código Fuente al Hipervector Semántico mediante UnixCoder

LAB

September 2026

## Resumen

La idea es mostrar la formalización conceptual del proceso de de código fuente en el modelo **UniXcoder**, sintetizando las notas manuscritas de la pizarra con los principios teóricos del modelo. Se detalla el flujo completo desde la extracción del Árbol de Sintaxis Abstracta (AST) mediante **Tree-sitter**, pasando por la función de mapeo biyectiva  $\mathcal{F}$  para el aplanado 1D, la codificación contextual mediante redes Transformer de 12 capas, hasta la consolidación del hipervector semántico  $\tilde{h} \in \mathbb{R}^{768}$  mediante *Mean Pooling*. Despues minimamente detallado el proceso de Similitud de Coseno, que sera nuestra metrica a la hora de comparar similitud conceptual/semantica entre dos scripts.

## 1. Observaciones

- A) En los calculos de complejidad se contempla que el pool de candidatos extraidos de la fase V0 se encuentra previamente vectorizado.
- B) De manera premiliminar puedo decir que los primero tres pasos del pipeline potencialmente pueden resolver el problema de seleccion de tags de la fase V0

## 2. Visión General del Pipeline

El procesamiento de código en **UniXcoder** resuelve el conflicto fundamental entre la estructura jerárquica de dos dimensiones de un programa (el árbol sintáctico) y la naturaleza secuencial unidimensional de las arquitecturas Transformer. El flujo de datos sigue seis etapas clave:

### Flujo Global de Procesamiento

|                    |                                                           |                   |                                                             |                         |                                                    |
|--------------------|-----------------------------------------------------------|-------------------|-------------------------------------------------------------|-------------------------|----------------------------------------------------|
| Código Fuente      | $\xrightarrow[\mathcal{O}(n)]{\text{Tree-sitter}}$        | AST (2D)          | $\xrightarrow[\mathcal{O}(V)]{\text{Función } \mathcal{F}}$ | AST Flattener (1D)      | $\xrightarrow{\text{Inyección Metadata}}$          |
| Input Formateado   | $\xrightarrow[\mathcal{O}(n^2)]{\text{Transformer } 12L}$ | Matriz $H_N^{12}$ | $\xrightarrow[\mathcal{O}(k)]{\text{Mean Pooling}}$         | Hipervector $\tilde{h}$ | $\xrightarrow[\mathcal{O}(1)]{\text{Cosine Sim.}}$ |
| Búsqueda/Similitud |                                                           |                   |                                                             |                         |                                                    |

## 3. Desglose Técnico de las Fases del Pipeline

### 3.1. Fase 1: Extracción del AST (Tree-sitter)

- **Entrada:** Fragmento de código fuente (*code snippet*) de longitud  $n$  en número de caracteres.
- **Proceso:** El parser determinista **Tree-sitter** realiza el análisis sintáctico (*syntax analysis*) generando un árbol jerárquico.
- **Complejidad Temporal:**  $\mathcal{O}(n)$ , proporcional al número de caracteres del código fuente.

### 3.2. Fase 2: AST y Estado de Contención Gramatical

- **Naturaleza del AST:** Como señala la pizarra, el AST representa un *estado de contención gramatical*, no un espacio semántico continuo. Captura reglas sintácticas, jerarquías de bloques y tipos de sentencias.
- **Inyección de Metadata y Reducción de Dimensionalidad:**
  1. Se inyecta metadata del sistema y se formatean las posiciones del código en los nodos.
  2. Para ingresar la estructura arbórea a la red, se aplica la función de mapeo biyectiva  $\mathcal{F}$  que aplanar el árbol a una **secuencia unidimensional** ( $AST_{1D}$ ).
  3. **Algoritmo de Mapeo 1-a-1 ( $\mathcal{F}$ ):** Inserta los delimitadores equivalentes  $\langle node\_name, left \rangle$  y  $\langle node\_name, right \rangle$  rodeando a los hijos de cada nodo.
- **Complejidad Temporal del Aplanado:**  $\mathcal{O}(V)$ , donde  $V$  representa el número total de nodos en el AST.

### 3.3. Fase 3: Formateo Final para UniXcoder

Una vez que el AST se ha transformado en  $AST_{1D}$ , se ensambla la secuencia de entrada concatenando el prefijo de comportamiento, el comentario descriptivo en lenguaje natural y la estructura aplanada:

$$\text{Input Formateado} = [\text{Enc}] + [\text{Comment}] + [AST_{1D}] \quad (1)$$

- **Token Prefijo [Enc]:** Configura la matriz de máscara de atención  $M$  ( $M_{ij} = 0$ ) para activar el modo de codificación bidireccional, permitiendo atención libre entre código, comentarios y estructura sintáctica.
- **Límite de Secuencia:** La longitud total de la secuencia formateada  $n = \# \text{Tokens en Format}(AST_{1D})$  se acota a  $n \leq 1024$  tokens (límite de hiperparámetros de preentrenamiento).
- **Complejidad de Tokenización BPE:**  $\mathcal{O}(S)$ , donde  $S$  es la longitud en caracteres de la secuencia concatenada.

### 3.4. Fase 4: Procesamiento Transformer y Matriz $H_N^{12}$

- **Operación en la Red:** La secuencia formateada ingresa al Transformer de 12 capas con una dimensión oculta  $d_h = 768$ .
- **Matriz Resultante:** Se genera la matriz de estados ocultos finales:

$$H_N^{12} \in \mathbb{R}^{n \times 768} \quad (2)$$

donde cada fila representa un vector de dimensión 768 enriquecido contextual y bidireccionalmente.

- **Complejidad Temporal de la Red:**  $\mathcal{O}(n^2 \cdot d_h) = \mathcal{O}(n^2)$  debido al cálculo de la matriz de autoatención de  $n \times n$ .

### 3.5. Fase 5: Generación del Hipervector Semántico (*Mean Pooling*)

Para colapsar la matriz  $n \times 768$  en un único vector representativo del fragmento de código:

- **Fórmula de Agregación:** Se aplica un promediado (*mean pooling*) sobre las posiciones  $k$  pertenecientes a los tokens de código/AST:

$$\tilde{h} = \frac{1}{k} \sum_{j \in \text{Code}} h_j^{12} \in \mathbb{R}^{768} \quad (3)$$

- **Propiedades del Hipervector  $\tilde{h}$ :** Condensa la información de la red, la estructura sintáctica del AST y la semántica del comentario en un espacio denso continuo de 768 dimensiones.
- **Complejidad Temporal:**  $\mathcal{O}(k \cdot 768) = \mathcal{O}(k)$ , donde  $k \leq n$  es la cantidad de tokens de código.

### 3.6. Fase 6: Búsqueda y Comparación por Similitud de Coseno

Para tareas como *Code Search*, *Clone Detection* o *Zero-shot Code-to-Code Search*:

$$\text{Cosine Similarity}(\tilde{h}_A, \tilde{h}_B) = \frac{\tilde{h}_A \cdot \tilde{h}_B}{\|\tilde{h}_A\|_2 \|\tilde{h}_B\|_2} \quad (4)$$

- **Complejidad de Comparación Directa:**  $\mathcal{O}(768) = \mathcal{O}(1)$  operaciones de producto punto entre dos hipervectores.

## 4. Resumen de Complejidades Temporales

La siguiente tabla resume los órdenes de complejidad de cada etapa del pipeline.

| Etapas del Pipeline       | Operación Principal                                | Complejidad        | Parámetro Clave                     |
|---------------------------|----------------------------------------------------|--------------------|-------------------------------------|
| 1. Extracción AST         | Análisis sintáctico ( <b>Tree-sitter</b> )         | $\mathcal{O}(n)$   | $n = \text{\#caracteres código}$    |
| 2. Aplanado $\mathcal{F}$ | Mapeo 1-a-1 de árbol a 1D                          | $\mathcal{O}(V)$   | $V = \text{\#nodos del AST}$        |
| 3. Tokenización BPE       | Conversión a IDs con tokens especiales             | $\mathcal{O}(S)$   | $S = \text{longitud de texto}$      |
| 4. Capas Transformer      | Autoatención bidireccional (12L)                   | $\mathcal{O}(n^2)$ | $n = \text{tokens input} \leq 1024$ |
| 5. Mean Pooling           | Promediado de vectores contextuales                | $\mathcal{O}(k)$   | $k = \text{tokens de código}$       |
| 6. Similitud Coseno       | Producto punto de hipervectores $\mathbb{R}^{768}$ | $\mathcal{O}(1)$   | Dimensión fija (768)                |

Cuadro 1: Complejidades temporales por instancia del proceso.

## 5. Conclusión

### Análisis de Escala de Complejidad Global:

Debido a la cota fija de 512 tokens establecida por la red (lo que impone un límite de aproximadamente 2000 bytes de código fuente de entrada) y a la dimensionalidad fija del tensor semántico ( $\mathbb{R}^{768}$ ), los factores individuales dependientes del tamaño de entrada  $n$  quedan acotados por constantes de valor superior.

Como consecuencia de estas restricciones dimensionales y de entrada, la complejidad temporal por instancia de código —que teóricamente corresponde a  $\mathcal{O}(n^2)$  debido al mecanismo de autoatención— pasa a operar en tiempo constante  $\mathcal{O}(1)$  por candidato procesado. Por lo tanto, la complejidad temporal global del pipeline pasa a escalar únicamente en función de la cantidad  $k$  de candidatos extraídos, resultando en una complejidad final estimada de:

$$\mathcal{O}(k)$$

**Nota:** Este cálculo excluye la fase inicial de búsqueda y extracción de snippets verosímiles dentro de los directorios candidatos realizada a través de la API de GitHub.